CloudFormation and Bicep

Write and operate cloud-native infrastructure as code: AWS CloudFormation and Azure Bicep, change previews, stacks, modules, testing and debugging failed deployments.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Native IaC: CloudFormation, Bicep or Terraform (12 min)
2. CloudFormation templates, parameters and resources (13 min)
3. Intrinsic functions, conditions and mappings (12 min)
4. Stacks, change sets and safe updates (13 min)
5. Nested stacks, StackSets and reusable modules (12 min)
6. Drift, rollbacks and resource protection (12 min)
7. Bicep syntax, resources and parameters (13 min)
8. Bicep modules, loops and conditions (12 min)
9. Bicep deployments, what-if and scopes (13 min)
10. Deployment stacks and delivery pipelines (12 min)
11. Testing, linting and policy checks for templates (12 min)
12. Debugging failed deployments and moving to Terraform (13 min)
Cheat sheet

Lesson 1. Native IaC: CloudFormation, Bicep or Terraform

12 minutes. Compare AWS CloudFormation and Azure Bicep with Terraform, and decide when a cloud's own infrastructure-as-code tool is the better choice.

You can describe infrastructure in code with your cloud's own language or with a neutral tool. The choice affects who holds the state, how fast new services are supported and how portable your skills are.

What you will learn

- What CloudFormation and Bicep are
- Where state lives and who manages it
- Strengths and limits against Terraform
- How to choose

Two native tools

CloudFormation reads a template (YAML or JSON) describing resources and creates them as a **stack**. **Bicep** is a cleaner language that compiles to an ARM template; Azure Resource Manager then creates the resources in a **resource group**, subscription or other scope. In both, the cloud service tracks what exists, so there is **no state file** for you to store, lock or lose.

	CloudFormation	Bicep	Terraform
Clouds	AWS	Azure	Many
Language	YAML or JSON	Bicep (compiles to ARM)	HCL
State	Held by AWS	Held by Azure	Your backend
New services	Usually day one (some lag)	Day one	Provider release needed
Cost	Free	Free	Free (plus paid options)

Strengths and limits

Native tools get new features early, integrate with the cloud's permissions and rollback, and need no backend setup. They can only manage that cloud, have fewer community modules, and the languages are tied to that vendor. Terraform shines for multi-cloud and non-cloud providers (DNS, monitoring, GitHub).

Choosing

Use the native tool when your estate is mostly one cloud and your team wants less to operate. Use Terraform when several clouds or third-party services are managed together. Many organisations use both: native for the cloud foundation, Terraform for the rest. The ideas (declare desired state, preview, apply, detect drift) carry across all three.

Diagram: Where each tool keeps its state

Part	What it does	What goes wrong
Template in Git	Desired state written as code.	Unreviewed changes go straight to production.
CloudFormation	AWS service creates and tracks a stack.	Stuck stacks can need manual rollback.
Bicep and ARM	Azure Resource Manager tracks deployments.	Complete mode can delete resources not in the file.
Terraform	Your tool plans and applies changes.	A lost or corrupted state file breaks plans.
Cloud resources	The real servers, networks and databases.	Manual console changes cause drift.

Flow: Template in Git to CloudFormation / Template in Git to Bicep and ARM / Template in Git to Terraform / CloudFormation to Cloud resources / Bicep and ARM to Cloud resources / Terraform to Cloud resources

MARGA by EknathaLabs marga.eknathalabs.com

Incident: The corrupted state file

> **Incident:** *A Terraform team lost a day when the state became corrupt; the native team had no state to lose.*

```
$ terraform plan
Error: state data in S3 does not have the expected content
$ aws cloudformation describe-stacks --stack-name net
StackStatus: UPDATE_COMPLETE
```

The native approach removed an entire failure class, but it does not remove drift or bad changes. The team standardised on native tools for single-cloud foundations and kept Terraform for cross-cloud pieces.

Common mistakes

- Choosing a tool for fashion rather than for the estate
- Assuming no state means no drift
- Mixing tools on the same resources
- Ignoring that skills in one language transfer partly to the other

Check yourself

Where does CloudFormation keep its state?

Answer: In the AWS service

What does Bicep compile to?

Answer: An ARM template

When is Terraform the better pick?

Answer: Managing several clouds and providers

What do all three share?

Answer: Declared desired state with preview and apply

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. CloudFormation templates, parameters and resources

13 minutes. Write a CloudFormation template with parameters, resources, outputs and references, and deploy it as a stack from the CLI.

A CloudFormation template is a document with a few sections. Once you know what each does you can read any template and write simple ones from the resource reference.

What you will learn

- Template sections
- Resources, properties and logical ids
- Parameters and outputs
- Deploying with the CLI

A small template

```
AWSTemplateFormatVersion: "2010-09-09"
Parameters:
  Env:
    Type: String
    AllowedValues: [dev, prod]
Resources:
  Logs:
    Type: AWS::S3::Bucket
    Properties:
      VersioningConfiguration:
        Status: Enabled
      Tags:
        - Key: env
          Value: !Ref Env
Outputs:
  BucketName:
    Value: !Ref Logs
    Export:
      Name: !Sub "${AWS::StackName}-bucket"
```

Each resource has a **logical id** (`Logs`) used inside the template and a **Type**. CloudFormation picks the physical name unless you set one; leave it unset so a stack can be created twice and resources can be replaced safely.

Sections

Section	Purpose
Parameters	Inputs with type, default and allowed values
Resources	The only required section
Outputs	Values to show or export for other stacks
Conditions, Mappings	Branching and lookup tables

Deploy

```
$ aws cloudformation validate-template --template-body file://logs.yaml
$ aws cloudformation deploy --template-file logs.yaml --stack-name logs-dev \
    --parameter-overrides Env=dev
$ aws cloudformation describe-stack-events --stack-name logs-dev
```

`deploy` creates or updates and shows the events. Read the **events** list when something fails: the first failed event holds the real reason.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: From template to resources

Part	What it does	What goes wrong
Template	YAML with parameters, resources and outputs.	A typo in a property name fails only at deploy time.
validate-template	Checks syntax, not whether values are valid.	Passing validation does not guarantee deployment.
Stack	The managed set of resources.	Failed create rolls back and leaves a ROLLBACK state.
Resources	Buckets, roles, networks created in dependency order.	Hard-coded global names collide across accounts.
Outputs	Values for people or other stacks.	An export in use blocks deleting its stack.

Flow: Template to validate-template / validate-template to Stack / Stack to Resources / Stack to Outputs

Incident: The bucket name that already existed

Incident: *A template with a fixed bucket name failed in the second account.*

```
$ aws cloudformation describe-stack-events --stack-name logs-prod
Logs  CREATE_FAILED  acme-logs already exists
logs-prod  ROLLBACK_COMPLETE
```

S3 names are global, so the second deploy could never use the same name. The fix was to remove the fixed name and let CloudFormation generate it, exposing the generated name through an output.

Common mistakes

- Hard-coding physical names of global resources
- Ignoring stack events when a deploy fails
- Treating validate-template as a full test
- Editing resources by hand in the console

Check yourself

What is required in a template?

Answer: Resources

What does Ref return for a bucket?

Answer: Its name

Where to look when a stack fails?

Answer: The stack events

Why avoid fixed physical names?

Answer: Replacement and multiple copies need unique names

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Intrinsic functions, conditions and mappings

12 minutes. Use Ref, GetAtt, Sub, Join, Select, If, Conditions and Mappings to make one CloudFormation template adapt to many environments.

Static templates are brittle. Intrinsic functions let a template refer to other resources and compute strings, while conditions and mappings capture the differences between environments in one file.

What you will learn

- Ref, GetAtt and Sub
- Join, Select, Split and ImportValue
- Conditions and If
- Mappings and SSM parameters

Functions you use daily

```
Resources:
  Role:
    Type: AWS::IAM::Role
    Properties:
      RoleName: !Sub "${AWS::StackName}-${Env}-app"
  Fn:
    Type: AWS::Lambda::Function
    Properties:
      Role: !GetAtt Role.Arn
      Environment:
        Variables:
          BUCKET: !ImportValue shared-bucket
```

- `!Ref` returns a parameter value or a resource's main id.
- `!GetAtt Res.Attr` returns another attribute, such as an ARN.
- `!Sub` substitutes `${...}` inside a string, including pseudo parameters like `${AWS::Region}`.
- `!Join`, `!Select`, `!Split` build and cut lists and strings.
- `!ImportValue` reads another stack's export.

Conditions

```
Conditions:
  IsProd: !Equals [!Ref Env, prod]
Resources:
  Alarm:
    Type: AWS::CloudWatch::Alarm
    Condition: IsProd
    Properties:
      Threshold: 80
```

A resource with a false condition is simply not created. `!If [IsProd, 3, 1]` picks a value.

Mappings and SSM

```
Mappings:
  Size:
    dev:  { Type: t3.small }
    prod: { Type: m5.large }
Parameters:
  Image:
    Type: AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>
    Default: /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64
```

Use `!FindInMap [Size, !Ref Env, Type]` to look up a value. The SSM parameter type resolves the latest image id for the current region at deploy time.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: One template, many environments

Part	What it does	What goes wrong
One template	Contains functions, conditions and mappings.	Too many branches make the template hard to read.
Parameters	Env and region choose the path.	A wrong value deploys the wrong sizing.
Mappings	Lookup table for sizes and settings.	A missing key fails the deploy.
Conditions	Decide whether a resource exists.	A condition on a used resource breaks references.
Resources	The final set of resources for that environment.	Differences should be reviewed with a change set.

Flow: One template to Parameters / One template to Mappings / One template to Conditions / Parameters to Resources / Mappings to Resources / Conditions to Resources

Incident: The AMI from another region

Incident: *A template with a fixed image id failed in a new region.*

```
$ aws cloudformation describe-stack-events --stack-name app-eu
Server  CREATE_FAILED  The image id 'ami-0abc1234' does not exist
region: eu-west-1  (template written for us-east-1)
```

Image ids differ per region. Replacing the fixed id with the SSM parameter type made the template portable and always current.

Common mistakes

- Hard-coding region-specific ids
- Writing one template per environment
- Overusing conditions until nothing is readable
- Referencing a conditional resource without guarding the reference

Check yourself

Which function returns a resource ARN?

Answer: GetAtt

What does a false condition do to a resource?

Answer: It is not created

What are mappings used for?

Answer: Lookup tables

Why use SSM parameter types for AMIs?

Answer: They resolve the right image per region

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 4. Stacks, change sets and safe updates

13 minutes. Preview updates with change sets, understand replacement versus in-place changes, and protect critical resources during stack updates.

CloudFormation updates are safe when you preview them and dangerous when you do not. Each property has an update behaviour, and some force a replacement that destroys data.

What you will learn

- Update behaviours
- Creating and reading a change set
- Retention policies and stack policies
- Rollback and stuck stacks

Update behaviours

Every property has one of three behaviours: **no interruption** (changed in place), **some interruption** (restart or brief outage), or **replacement** (a new resource is created, then the old one deleted). Replacement of a database or volume loses data unless protected.

Change sets

```
$ aws cloudformation create-change-set --stack-name app --change-set-name c1 \
    --template-body file://app.yaml --parameters ParameterKey=Env,ParameterValue=prod
$ aws cloudformation describe-change-set --stack-name app --change-set-name c1 \
    --query 'Changes[].ResourceChange.[Action,LogicalResourceId,Replacement]'
[["Modify","Database","True"]]
$ aws cloudformation execute-change-set --stack-name app --change-set-name c1
```

Read each row: **Action** (Add, Modify, Remove) and **Replacement** (True, False, Conditional). Treat True or Remove on stateful resources as a stop sign.

Protection

- `DeletionPolicy: Retain` or `Snapshot` keeps or snapshots a resource when the stack removes it.
- `UpdateReplacePolicy: Snapshot` snapshots the old resource when replaced.
- **Termination protection** blocks deleting the stack.
- A **stack policy** denies updates to chosen resources unless explicitly overridden.

When updates fail

By default a failed update **rolls back** to the previous state. A failed rollback leaves `UPDATE_ROLLBACK_FAILED`; fix the cause (often a resource changed by hand) and continue the rollback, skipping resources only as a last resort.

Diagram: A safe update

Part	What it does	What goes wrong
Edit the template	Change a property in Git.	Some properties force replacement.
Create change set	Shows what would change without doing it.	Skipping it hides replacement.
Review	A person reads Action and Replacement.	Approving without reading defeats the purpose.
Execute	CloudFormation applies the changes.	A failure triggers rollback.
Retention policies	Snapshot or retain stateful resources.	Without them replacement loses data.

Flow: Edit the template to Create change set / Create change set to Review / Review to Execute / Retention policies to Execute

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: The database that was recreated

Incident: *A small property change replaced a database with an empty one.*

```
$ aws cloudformation describe-change-set --change-set-name c1
Modify  Database  Replacement: True
stack status: UPDATE_COMPLETE
select count(*) from orders;  0
```

The update was run directly with no change set and no snapshot policy. The team restored from backup, added DeletionPolicy and UpdateReplacePolicy Snapshot, and made change-set review part of the pipeline.

Common mistakes

- Updating production without a change set
- No retention policy on databases and volumes
- Editing resources by hand, causing failed rollbacks
- Skipping resources in a rollback without understanding why

Check yourself

What does Replacement True mean?

Answer: A new resource is made and the old one removed

What does UpdateReplacePolicy Snapshot do?

Answer: Snapshots the old resource when it is replaced

What does termination protection block?

Answer: Deleting the stack

What is the first step before executing an update?

Answer: Read the change set

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Nested stacks, StackSets and reusable modules

12 minutes. Split large templates into nested stacks, roll one template out to many accounts and regions with StackSets, and share patterns as modules.

As an estate grows, one template cannot hold it. You need ways to split work by layer and ways to repeat a baseline across many accounts and regions.

What you will learn

- Nested stacks and their limits
- Cross-stack references and their coupling
- StackSets for many accounts and regions
- Modules and the registry

Splitting

A **nested stack** is a stack created by a resource of type `AWS::CloudFormation::Stack` in a parent template, with `TemplateURL` pointing to a template in S3 and `Parameters` passing values down. A single stack is limited to 500 resources, so nesting helps both size and organisation.

```
Resources:
  Network:
    Type: AWS::CloudFormation::Stack
    Properties:
      TemplateURL: https://s3.amazonaws.com/acme-templates/network.yaml
      Parameters: { Env: !Ref Env }
```

Cross-stack references

`Export` and `!ImportValue` link separate stacks. An export in use **cannot be changed or deleted**, which couples the stacks. A looser option writes values to SSM Parameter Store and reads them as parameters, so layers (network, data, app) can change independently.

StackSets

A **StackSet** deploys one template to many accounts and regions. With AWS Organizations and service-managed permissions, new accounts in an organisational unit can receive the stack automatically. Set operation preferences: maximum concurrent accounts and a failure tolerance, so a bad template stops after a few accounts instead of all.

Modules

CloudFormation **modules** package a resource pattern (a compliant bucket) that teams reuse from the registry. Higher-level options, such as the CDK, generate templates from code.

Diagram: Splitting and repeating templates

Part	What it does	What goes wrong
Parent template	Passes parameters to child stacks.	Child failures roll back the parent.
Network stack	VPC and subnets.	Changing an exported value is blocked while imported.
App stack	Services that read network outputs.	Tight exports slow changes down.
StackSet	Same baseline in many accounts and regions.	No failure tolerance pushes a bad change everywhere.
Member accounts	Receive the stack automatically.	Drifted accounts fall out of date.

Flow: Parent template to Network stack / Parent template to App stack / StackSet to Member accounts

MARGA by EknathaLabs
marga.eknathalabs.com

Incident: Locked by an export

Incident: *A team could not change a network output because another stack imported it.*

```
$ aws cloudformation update-stack --stack-name net --template-body file://net.yaml
An error occurred: Cannot update export net-vpc as it is in use by app
imports: app-dev, app-prod
```

The export tied the layers together. The team moved shared values to SSM parameters read at deploy time, which let the network and app stacks change independently.

Common mistakes

- Using exports for everything
- StackSets with no failure tolerance
- Nested templates not versioned in S3
- Large single templates near the resource limit

Check yourself

What resource type creates a nested stack?

Answer: AWS

What does a StackSet do?

Answer: Deploys a template to many accounts and regions

Why can an export be a problem?

Answer: It cannot change while imported

Why set a StackSet failure tolerance?

Answer: To stop a bad change after a few accounts

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Drift, rollbacks and resource protection

12 minutes. Detect drift between a stack and reality, import existing resources, handle stuck rollbacks and protect data from accidental deletion.

Infrastructure as code is only true while nobody edits the resources by hand. Drift detection, import and protection rules keep the code and the cloud in agreement.

What you will learn

- Detecting and reading drift
- Importing resources into a stack
- Stuck stack states and how to recover
- Guardrails against manual changes

Detect drift

```
$ aws cloudformation detect-stack-drift --stack-name net
$ aws cloudformation describe-stack-resource-drifts --stack-name net \
    --stack-resource-drift-status-filters MODIFIED DELETED
```

The report lists each resource as `IN_SYNC`, `MODIFIED` or `DELETED`, with the expected and actual property values. Not every resource type supports drift detection; check the list. Decide each case: change the template to accept the new value, or update the stack to restore the old one.

Import

To bring an existing resource under management, add it to the template with a `DeletionPolicy` and run a **resource import** change set with the resource's identifier. Afterwards the stack manages it.

Stuck states

Status	Meaning	Action
`ROLLBACK_COMPLETE`	A create failed and rolled back	Delete the stack and deploy again
`UPDATE_ROLLBACK_FAILED`	Rollback itself failed	Fix the cause, then continue update rollback
`DELETE_FAILED`	A resource could not be deleted (non-empty bucket)	Empty or retain it, then delete

Guardrails

Restrict console write access with IAM, use stack policies for critical resources, enable CloudTrail to find who changed a resource and AWS Config rules to alert on forbidden changes. Create a pipeline as the only path to production.

Diagram: Keeping code and cloud in agreement

Part	What it does	What goes wrong
Template in Git	The intended state.	Out-of-date code misleads the next engineer.
Pipeline deploys	Only path that changes production.	Console access bypasses it.
Resources	What actually exists.	Manual edits cause drift.
Drift detection	Compares resources to the template on a schedule.	Unscheduled checks find drift too late.
CloudTrail	Records who changed what.	Without it, drift has no owner.

Flow: Template in Git to Pipeline deploys / Pipeline deploys to Resources / Resources to Drift detection / Drift detection to CloudTrail

MARGA by EknathaLabs marga.eknathalabs.com

Incident: The open security group

Incident: *A debugging change in the console left a rule open to the internet for months.*

```
$ aws cloudformation describe-stack-resource-drifts --stack-name net
SecurityGroup  MODIFIED
IpPermissions[0].CidrIp  expected 10.0.0.0/16  actual 0.0.0.0/0
```

Drift detection found it. The team re-applied the stack, scheduled weekly drift checks and removed console write access outside an emergency role.

Common mistakes

- Fixing problems in the console and not in the template
- Never running drift detection
- Deleting a stack that still holds data without a retention policy
- Skipping resources in a failed rollback

Check yourself

What does MODIFIED mean in a drift report?

Answer: A property no longer matches the template

How do you bring an existing resource under CloudFormation?

Answer: Resource import

What do you do with ROLLBACK_COMPLETE?

Answer: Delete the stack and deploy again

Which service records who made a console change?

Answer: CloudTrail

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Bicep syntax, resources and parameters

13 minutes. Write Bicep files with parameters, variables, resources, outputs and decorators, and deploy them to a resource group.

Bicep removes the noise of ARM JSON. A file has parameters in, resources in the middle and outputs out, and the compiler handles the order of creation.

What you will learn

- The parts of a Bicep file
- Decorators, types and secure values
- Dependencies and references
- Deploying and converting

A small file

```
@description('Short environment name')
@allowed(['dev', 'prod'])
param env string = 'dev'

@secure()
param adminPassword string

var storageName = 'st${uniqueString(resourceGroup().id)}${env}'

resource sa 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: storageName
  location: resourceGroup().location
  sku: { name: env == 'prod' ? 'Standard_GRS' : 'Standard_LRS' }
  kind: 'StorageV2'
  properties: { minimumTlsVersion: 'TLS1_2' }
}

output blobEndpoint string = sa.properties.primaryEndpoints.blob
```

- `param` is an input; `@allowed`, `@secure`, `@minLength` constrain it.
- `var` is a computed value.
- `resource` has a symbolic name, a type with an API version, and properties. Referring to `sa.properties...` creates an **implicit dependency**, so Azure orders the work.
- `uniqueString()` builds a stable, unique suffix for global names.

Deploy

```
$ az group create --name rg-app-dev --location westeurope
$ az deployment group create --resource-group rg-app-dev --template-file main.bicep \
    --parameters env=dev adminPassword=$PW
$ az bicep build --file main.bicep       # compile to ARM JSON
$ az bicep decompile --file old.json     # convert ARM to Bicep
```

Use `.bicepparam` files to keep environment values out of the template.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Bicep to Azure

Part	What it does	What goes wrong
main.bicep	Parameters, variables, resources and outputs.	Old API versions miss new features.
Bicep compiler	Checks types and produces an ARM template.	Errors appear before any deployment.
Azure Resource Manager	Orders and creates the resources.	Failed resources leave a partial deployment.
Resource group	Holds the created resources.	Deleting it removes everything inside.
Parameter file	Values for each environment.	Secrets in plain files leak.

Flow: main.bicep to Bicep compiler / Bicep compiler to Azure Resource Manager / Azure Resource Manager to Resource group / Parameter file to Azure Resource Manager

Incident: The name that was not unique

Incident: *A storage account deploy failed because the name was already taken.*

```
$ az deployment group create --resource-group rg-app-dev --template-file main.bicep
(StorageAccountAlreadyTaken) The storage account named acmestorage is already taken.
```

Storage account names are global. Building the name with uniqueString() made it stable inside the resource group and unique across Azure.

Common mistakes

- Fixed names for globally unique resources
- Secrets as plain parameters instead of @secure or Key Vault references
- Very old API versions
- Editing generated ARM JSON instead of the Bicep file

Check yourself

What does @secure() do?

Answer: Hides a parameter value from logs and output

How does Bicep order resources?

Answer: By references between them

What does uniqueString help with?

Answer: Globally unique names

Which command compiles Bicep?

Answer: az bicep build

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Bicep modules, loops and conditions

12 minutes. Split Bicep into modules, repeat resources with for loops, switch them with conditions and share modules through a registry.

As a Bicep estate grows, copy and paste becomes the enemy. Modules give structure, loops remove repetition, and conditions keep one file useful for many environments.

What you will learn

- Modules and their inputs and outputs
- Loops over arrays and objects
- Conditions and existing resources
- Sharing modules via a registry

Modules

```
module net './modules/network.bicep' = {
  name: 'network'
  params: {
    vnetName: 'vnet-${env}'
    subnets: ['web', 'app', 'data']
  }
}

output vnetId string = net.outputs.vnetId
```

Each module is a Bicep file with `param` and `output`. The module runs as its own nested deployment, which appears under that name in the portal and in deployment history.

Loops and conditions

```
resource subnet 'Microsoft.Network/virtualNetworks/subnets@2023-09-01' = [for (name, i) in subnets \
    : {
  parent: vnet
  name: name
  properties: { addressPrefix: '10.0.${i}.0/24' }
}]

resource lock 'Microsoft.Authorization/locks@2020-05-01' = if (env == 'prod') {
  name: 'no-delete'
  properties: { level: 'CanNotDelete' }
}
```

Use `existing` to refer to a resource created elsewhere: `resource kv 'Microsoft.KeyVault/vaults@2023-07-01' existing = { name: kvName }`.

Registry

Publish tested modules to an **Azure Container Registry** with `az bicep publish`, and use them as `module m 'br:acme.azurecr.io/bicep/network:v1.2' = { ... }`. Public ready-made modules are available under the `br/public` alias. Version modules and pin versions.

Diagram: Modules and loops

Part	What it does	What goes wrong
main.bicep	Orchestrates modules with parameters.	Large mains become unreadable; keep it thin.
Network module	Creates a vnet and subnets by looping.	Changing a loop key can recreate subnets.
App module	Uses outputs from the network module.	Missing outputs break the dependency.
Module registry	Stores versioned shared modules in ACR.	Unpinned versions change under you.
Resource group	Result of all nested deployments.	Failures appear in the nested deployment name.

MARGA by EknathaLabs marga.eknathalabs.com

Flow: main.bicep to Network module / main.bicep to App module / Module registry to main.bicep / Network module to Resource group / App module to Resource group

Incident: Six copies of one vnet

Incident: *A subnet change had to be repeated in six files and one was missed.*

```
$ az network vnet subnet list --vnet-name vnet-eu -g rg-eu -o table
web  app
$ az network vnet subnet list --vnet-name vnet-us -g rg-us -o table
web  app  data
```

Copies drifted apart. The team created one network module with a subnet list, published it to a registry with a version, and every environment now uses it.

Common mistakes

- Copy and paste instead of modules
- Unpinned registry module versions
- Changing loop items so resources are recreated
- Modules with dozens of parameters

Check yourself

How does a Bicep module return values?

Answer: Through output

What does a Bicep if condition do?

Answer: Creates the resource only when true

What is the existing keyword for?

Answer: Referring to a resource created elsewhere

Why pin module versions?

Answer: Shared modules change over time

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Bicep deployments, what-if and scopes

13 minutes. Choose deployment scopes, preview changes with what-if, understand incremental and complete modes and keep deployment history manageable.

Where a deployment lands and how it changes existing resources matter as much as the template. Scope, mode and a preview step are the three controls.

What you will learn

- Scopes from resource group to tenant
- Incremental and complete modes
- What-if previews
- Deployment history and protection

Scopes

```
targetScope = 'subscription'

resource rg 'Microsoft.Resources/resourceGroups@2024-03-01' = {
  name: 'rg-app-${env}'
  location: 'westeurope'
}

module app './app.bicep' = {
  scope: rg
  name: 'app'
}
```

A file targets a **resource group** (default), **subscription** (create groups, policies, role assignments), **management group** or **tenant**. Modules can switch scope as shown. Use the matching command: `az deployment group create` or `az deployment sub create`.

Modes

Mode	Behaviour
Incremental (default)	Adds and updates resources in the file; leaves others alone
Complete	Also deletes resources in the group that are not in the file

What-if

```
$ az deployment group what-if -g rg-app-dev --template-file main.bicep --parameters env=dev
  + Create   Microsoft.Network/publicIPAddresses/pip-app
  ~ Modify   Microsoft.Storage/storageAccounts/stapp
      ~ sku.name: "Standard_LRS" => "Standard_GRS"
  - Delete   Microsoft.Compute/virtualMachines/vm-old
```

Read the symbols: `+` create, `~` modify, `-` delete, `=` no change. Put what-if in pull request checks and require approval of the output for production.

History and protection

Azure keeps up to 800 deployments per group and can delete the oldest automatically. Add **resource locks** (CanNotDelete) and **Azure Policy** to block risky changes regardless of the template.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: A safe Azure deployment

Part	What it does	What goes wrong
Pull request	Bicep change reviewed by a person.	A skipped review sends untested code on.
what-if	Lists creates, changes and deletes.	Ignoring deletes is the classic mistake.
Approval	Someone confirms the preview for production.	Rubber stamping defeats the check.
Deployment	Applies in incremental mode.	Complete mode can delete unlisted resources.
Locks and Policy	Block deletes and unsafe settings regardless of code.	Missing locks leave critical resources open to deletion.

Flow: Pull request to what-if / what-if to Approval / Approval to Deployment / Locks and Policy to Deployment

Incident: Complete mode deleted the lot

Incident: *A partial template ran in complete mode and removed every resource not in it.*

```
$ az deployment group create -g rg-app --template-file part.bicep --mode Complete
Deleted: Microsoft.Compute/virtualMachines/vm-db
Deleted: Microsoft.Storage/storageAccounts/stdata
```

A what-if would have shown the deletes. The team restored from backup, set CanNotDelete locks on data resources and made what-if output a required step in the pipeline.

Common mistakes

- Complete mode with a template that is not the full truth
- Skipping what-if on production
- No resource locks on data stores
- Letting deployment history hit its limit

Check yourself

What does what-if show?

Answer: Planned creates, modifies and deletes

Which mode deletes unlisted resources?

Answer: Complete

Which scope creates resource groups?

Answer: Subscription

What do CanNotDelete locks do?

Answer: Block deleting a resource

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 10. Deployment stacks and delivery pipelines

12 minutes. Manage resource lifecycles with Azure deployment stacks, and run CloudFormation and Bicep from CI pipelines with least-privilege roles.

A template is only as safe as its path to production. Deployment stacks fill a gap in Azure by tracking what a deployment owns, and a good pipeline replaces human keys with short-lived identity.

What you will learn

- Deployment stacks and deny settings
- Federated identity for pipelines
- Service roles and least privilege
- Environment promotion and approvals

Deployment stacks

A plain Azure deployment forgets what it created. A **deployment stack** keeps the list, so when a resource is removed from the template you choose whether to **detach** it or **delete** it. **Deny settings** can block changes to managed resources by anyone except the stack itself.

```
$ az stack group create --name app --resource-group rg-app --template-file main.bicep \
    --action-on-unmanage deleteResources --deny-settings-mode denyDelete
```

Pipelines without stored keys

Use **OpenID Connect federation**: the pipeline presents a short-lived token from the CI provider, and the cloud exchanges it for a role limited to the repository, branch or environment.

```
# GitHub Actions
permissions:
  id-token: write
  contents: read
steps:
  - uses: azure/login@v2
    with:
      client-id: ${{ vars.CLIENT_ID }}
      tenant-id: ${{ vars.TENANT_ID }}
      subscription-id: ${{ vars.SUB }}
  - run: az deployment group what-if -g rg-app --template-file main.bicep
```

For AWS, the pipeline assumes a role through an OIDC provider. For CloudFormation, pass a **service role** so the stack uses its own permissions rather than the caller's.

Promotion

Build once, version the template, deploy to dev, run checks, then promote to production behind a manual approval that displays the change set or what-if output. Keep separate identities and subscriptions or accounts per environment.

Diagram: A least-privilege pipeline

Part	What it does	What goes wrong
Git commit	Templates and parameters reviewed.	Unreviewed merges deploy straight to production.
Pipeline	Lints, previews and deploys.	A stored key in settings can leak.
Federated identity	Short-lived token exchanged for a narrow role.	A too-broad role undoes the benefit.
Dev environment	First target of every change.	Skipping dev finds problems in production.
Production	Deployed after approval of the preview.	No approval gate means no last check.

Flow: Git commit to Pipeline / Pipeline to Federated identity / Federated identity to Dev environment / Federated identity to Production

MARGA by EknathaLabs marga.eknathalabs.com

Incident: A key in the log

Incident: *A long-lived cloud key stored in pipeline settings was printed by a debugging step.*

```
build.log: echo $AZURE_CLIENT_SECRET
build.log: AZURE_CLIENT_SECRET=Q8x...
security: secret rotated; sign-ins reviewed
```

The key was valid for years and gave wide access. Federated credentials replaced it: nothing is stored, the token lives for minutes and only the main branch's production job can assume the production role.

Common mistakes

- Long-lived keys in CI settings
- One pipeline identity with owner rights everywhere
- No approval step for production
- Deployment stack deny settings that block the pipeline itself

Check yourself

What does a deployment stack track?

Answer: Resources the deployment created

What replaces stored cloud keys in a pipeline?

Answer: Federated OIDC identity

What is a CloudFormation service role?

Answer: A role the stack uses instead of the caller's

Why promote through environments?

Answer: Catch problems before production

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Testing, linting and policy checks for templates

12 minutes. Catch template mistakes before deployment with cfn-lint, cfn-guard, the Bicep linter, what-if, policy as code and ephemeral test environments.

A template change can open a firewall or delete a database. Automated checks catch most of the common mistakes at no cost, in seconds, before anyone deploys.

What you will learn

- Linting for syntax and best practice
- Policy as code for standards
- Previews and security scanning
- Test environments and smoke tests

The layers

Layer	CloudFormation	Bicep
Syntax and best practice	`cfn-lint`	`az bicep build`, Bicep linter
Policy rules	`cfn-guard`	PSRule for Azure, Azure Policy
Security scan	Checkov, Trivy	Checkov, Trivy
Preview	Change set	What-if
Real test	Deploy to a test account	Deploy to a test subscription

Examples

```
$ cfn-lint template.yaml
E3012 Property Resources/Logs/Properties/Versioning should be of type Object
$ cfn-guard validate --data template.yaml --rules s3-encryption.guard
Logs/BucketEncryption: FAIL (rule: s3_bucket_encryption_enabled)
$ az bicep build --file main.bicep
Warning no-hardcoded-env-urls: Environment URLs should not be hardcoded
```

A guard rule is a short statement of the standard, for example "every S3 bucket has encryption".

```
rule s3_bucket_encryption_enabled {
  Resources.*[ Type == 'AWS::S3::Bucket' ].Properties.BucketEncryption exists
}
```

Test deployments

Linters cannot prove that a stack really deploys, that an app responds or that permissions are enough. Create a short-lived environment in a sandbox account, run smoke tests, then delete it. Use **taskcat** for CloudFormation across regions. Always clean up to avoid cost.

Wiring

Run checks locally with pre-commit and again in CI as the required status, so a pull request cannot merge on failure.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Template checks in order

Part	What it does	What goes wrong
Pull request	Starts the checks.	Checks that run only on request are skipped.
Lint	Syntax and best practice.	Warnings left alone become habits.
Policy rules	Encryption, tags, public access rules.	Rules nobody owns go stale.
Preview	Change set or what-if.	Previewing against the wrong environment misleads.
Test deployment	Short-lived environment with smoke tests.	Forgotten test stacks cost money.

Flow: Pull request to Lint / Lint to Policy rules / Policy rules to Preview / Preview to Test deployment

Incident: Open to the world

Incident: *A pull request opened a security group to the internet; policy checks caught it.*

```
$ cfn-guard validate --data app.yaml --rules network.guard
sg-web: FAIL (rule: no_open_ssh)
ingress 0.0.0.0/0 on port 22
```

The check failed before merge. The author restricted the source range, the review took two minutes, and no auditor had to find it later.

Common mistakes

- Linting only on a developer laptop
- Treating warnings as noise
- Test stacks never cleaned up
- Policy rules with no owner

Check yourself

Which tool lints CloudFormation templates?

Answer: cfn-lint

What does cfn-guard check?

Answer: Policy rules over templates

Why still run a test deployment?

Answer: Tools cannot prove a stack really works

Where should checks be required?

Answer: As CI status on every pull request

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 12. Debugging failed deployments and moving to Terraform

13 minutes. Read CloudFormation events and Azure deployment errors, fix common failures and plan a safe migration to or from Terraform.

Failed deployments feel mysterious because the final status is vague. The details sit one level down, in the events or operations of the individual resources. A fixed method gets you there fast.

What you will learn

- Reading events and operations
- Common errors and fixes
- Cleaning up stuck or partial deployments
- Migrating between native tools and Terraform

CloudFormation

```
$ aws cloudformation describe-stack-events --stack-name app \
    --query 'StackEvents[?ResourceStatus==`CREATE_FAILED`].[LogicalResourceId,ResourceStatusReason \
    ]'
```

Find the **earliest** failed resource. Later `CREATE_FAILED` lines for dependants usually say "resource creation cancelled".

Azure

```
$ az deployment group show -g rg-app -n main --query properties.error
$ az deployment operation group list -g rg-app -n main \
    --query "[?properties.provisioningState=='Failed'].properties.statusMessage"
```

Open the operation for the failed resource; the top-level error often says only "deployment failed". Check the **Activity Log** for who attempted what.

Common causes

Message contains	Cause	Fix
AlreadyExists / AlreadyTaken	Name collision	Use generated or unique names
AccessDenied / AuthorizationFailed	Missing permission	Add the specific action to the role
LimitExceeded / QuotaExceeded	Account quota	Request an increase or change region
InvalidTemplate / ValidationError	Bad value or type	Run lint and what-if
DependsOn / ResourceNotFound	Order or wrong scope	Reference the resource to create a dependency

Migrating

To move to Terraform, write matching Terraform, then **import** each resource into Terraform state (`terraform import`, or `aztfexport` for Azure), confirm `terraform plan` shows no change, set `DeletionPolicy: Retain` in CloudFormation, then remove the stack. Moving the other way follows the same steps with `aws cloudformation` resource import. Migrate one layer at a time.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Finding the real error

Part	What it does	What goes wrong
Deployment failed	The final status gives no detail.	Reading only this line wastes time.
Events or operations	Per-resource log of what happened.	Later failures are often consequences.
First failed resource	The earliest failure holds the true reason.	A fix to later items does not help.
Cause category	Name, permission, quota, value or order.	Misreading the category sends you the wrong way.
CloudTrail or Activity Log	Shows who attempted the action and the denied permission.	Disabled logs leave no evidence.

Flow: Deployment failed to Events or operations / Events or operations to First failed resource / First failed resource to Cause category / Events or operations to CloudTrail or Activity Log

Incident: The permission nobody looked at

Incident: *A pipeline deployment failed with a generic message for two days.*

```
$ az deployment group show -g rg-app -n main --query properties.error.message
At least one resource deployment operation failed.
$ az deployment operation group list -g rg-app -n main
vnet  Failed  AuthorizationFailed: no permission for Microsoft.Network/virtualNetworks/write
```

The summary hid a simple permission gap. Adding the one missing action to the pipeline role fixed it, and the team added a runbook step: open the failed operation first.

Common mistakes

- Reading only the last error line
- Fixing later failures before the first
- Giving owner rights to fix a permission error
- Deleting a stack before importing its resources elsewhere

Check yourself

Which failed event do you read first?

Answer: The earliest

AuthorizationFailed usually means what?

Answer: The identity lacks a permission

What is the first step to move a stack to Terraform?

Answer: Write matching Terraform and import resources

Where do you see who made a change?

Answer: CloudTrail or the Activity Log

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: CloudFormation and Bicep

Commands to validate, preview, deploy and debug CloudFormation stacks and Bicep deployments, and the checks to run first.

CloudFormation

```
aws cloudformation validate-template --template-body file://app.yaml
cfn-lint app.yaml
aws cloudformation deploy --template-file app.yaml --stack-name app-dev --parameter-overrides Env= \
    dev
aws cloudformation describe-stack-events --stack-name app-dev --max-items 20
aws cloudformation describe-stacks --stack-name app-dev --query 'Stacks[0].StackStatus'
```

Preview an update

```
aws cloudformation create-change-set --stack-name app --change-set-name c1 --template-body file:// \
    app.yaml
aws cloudformation describe-change-set --stack-name app --change-set-name c1 \
  --query 'Changes[].ResourceChange.[Action,LogicalResourceId,Replacement]'
aws cloudformation execute-change-set --stack-name app --change-set-name c1
```

Drift

```
aws cloudformation detect-stack-drift --stack-name app
aws cloudformation describe-stack-resource-drifts --stack-name app --stack-resource-drift-status-f \
    ilters MODIFIED DELETED
```

Bicep

```
az bicep build --file main.bicep
az deployment group what-if --resource-group rg-app --template-file main.bicep --parameters env=de \
    v
az deployment group create --resource-group rg-app --template-file main.bicep --parameters env=dev
az deployment operation group list --resource-group rg-app --name main
```

Stack states

Status	What to do
ROLLBACK_COMPLETE	Read the first failed event, delete the stack, deploy again
UPDATE_ROLLBACK_FAILED	Fix the cause, then continue the rollback
DELETE_FAILED	Empty or retain the blocking resource, then delete

Safe habits

1. Run lint, policy checks and a preview on every pull request.
2. Read Replacement and Delete lines before approving.
3. Protect stateful resources with snapshot or retain policies.
4. Deploy from a pipeline with a federated identity, not a stored key.
5. Check drift on a schedule and fix it through code.

Safe deploys

```
aws cloudformation create-change-set --stack-name app --change-set-name c1 --template-body file:// \
    t.yaml

aws cloudformation describe-stack-events --stack-name app --max-items 10

az deployment group what-if -g rg-app -f main.bicep

cfn-lint t.yaml
```

Danger: *A property change can replace a resource, and replacing a database deletes its data. Read the change set* `Replacement` *column before you execute it.*

MARGA by EknathaLabs marga.eknathalabs.com

Warning: *Set* `DeletionPolicy: Retain` *or* `Snapshot` *on stateful resources before you delete or rename them.*

Warning: *Bicep complete mode deletes resources in the group that are not in the file. Run* `what-if` *first.*

Warning: *Editing resources by hand causes drift. The next update can undo your change or fail.*

MARGA by EknathaLabs
marga.eknathalabs.com